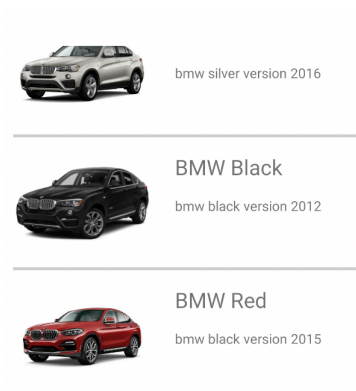


Class 12

使用 RecyclerView



1. 在佈局文件中加入 RecyclerView

```
<androidx.recyclerview.widget.RecyclerView  
    android:id="@+id/recyclerView"  
    android:layout_width="match_parent"  
    android:layout_height="500dp" />
```

2. 在 res->layout->New Layout Resource File-> 建立一個 RecyclerView 使用的單項模版



3. 根據模版內容建立一個單項 class

```
// car.kt  
data class Car(val name: String, val photo: Int, val desc: String)
```

4. 建立 Adapter

```
class CarAdapter(private val cars: List<Car>) :
RecyclerView.Adapter<CarAdapter.CarViewHolder>() {

    inner class CarViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView)
    {
        val photoImageView: ImageView = itemView.findViewById(R.id.imageview)
        val nameTextView: TextView = itemView.findViewById(R.id.nameTextView)
        val descTextView: TextView = itemView.findViewById(R.id.descTextView)
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
CarViewHolder {

        val view =
LayoutInflater.from(parent.context).inflate(R.layout.car_layout, parent, false)
        return CarViewHolder(view)
    }

    override fun onBindViewHolder(holder: CarViewHolder, position: Int) {
        holder.photoImageView.setImageResource(cars[position].photo)
        holder.nameTextView.text = cars[position].name
        holder.descTextView.text = cars[position].desc
    }

    override fun getItemCount() = cars.size
}

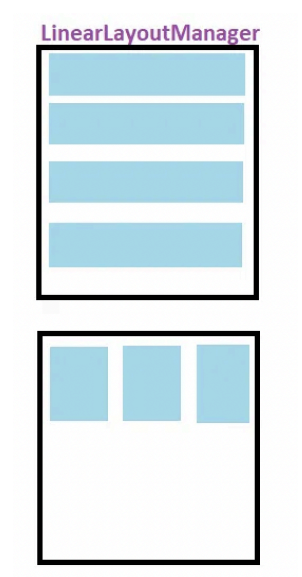
// cars: List<Car> RecyclerView 的數據來源
// CarViewHolder 建立一個內部類，得到單項模版中各個界面元素
// R.layout.car_layout 單項模版的文件名
// position 表示當前處理單項的 index， 由數據源 List中得到數據並由 RecyclerView 中呈
現
```

5. 為 RecyclerView 加入數據源和 Adapter

```
private val carList: ArrayList<Car> = arrayListOf(
    Car("BMW Black", R.drawable.car_bmv_black, "bmw black version 2012"),
    Car("BMW Red", R.drawable.car_bmv_red, "bmw black version 2015"),
    Car("BMW Silver", R.drawable.car_bmv_silver, "bmw silver version 2016")
)

recyclerView.adapter = CarAdapter(carList)
recyclerView.layoutManager = LinearLayoutManager(this)
```

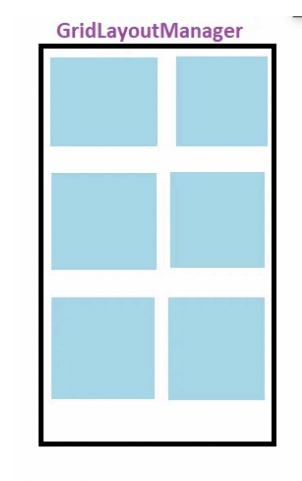
RecyclerView 設置不同的 Layout



```
val layoutManager = LinearLayoutManager(this)
recyclerView.layoutManager = layoutManager
```

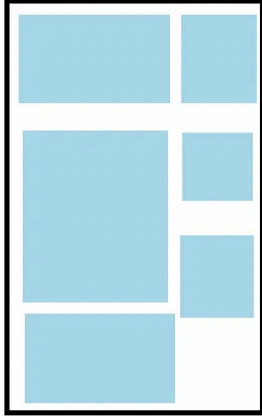
水平顯示

```
val layoutManager = LinearLayoutManager(this)
layoutManager.orientation = LinearLayoutManager.HORIZONTAL
recyclerView.layoutManager = layoutManager
```



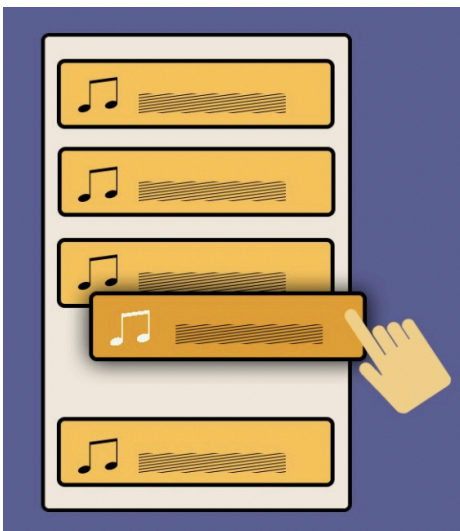
```
val layoutManager = GridLayoutManager(this, 2)
recyclerView.layoutManager = layoutManager
```

StaggeredGridLayoutManager



```
val layoutManager = StaggeredGridLayoutManager(2,  
StaggeredGridLayoutManager.VERTICAL)  
recyclerView.layoutManager = layoutManager
```

使 RecyclerView 支持拖拽操作



1.建立 itemTouchHelper

```
val itemTouchHelper = ItemTouchHelper(object :  
ItemTouchHelper.Callback() {  
    override fun onMove(  
        recyclerView: RecyclerView,  
        viewHolder: RecyclerView.ViewHolder,
```

```

        target: RecyclerView.ViewHolder
    ): Boolean {
        adapter.notifyItemMoved(viewHolder.adapterPosition,
target.adapterPosition)
        return true
    }

    override fun onSwiped(viewHolder: RecyclerView.ViewHolder,
direction: Int) {
    }

    override fun getMovementFlags(
        recyclerView: RecyclerView,
        viewHolder: RecyclerView.ViewHolder
    ): Int {
        val dragFlags = ItemTouchHelper.UP or ItemTouchHelper.DOWN
        // 不需要左右滑動
        val swipeFlags = 0

        return makeMovementFlags(dragFlags, swipeFlags)
    }

    override fun onSelectedChanged(viewHolder: RecyclerView.ViewHolder?,
actionState: Int) {
        super.onSelectedChanged(viewHolder, actionState)
        if(actionState == ItemTouchHelper.ACTION_STATE_DRAG) {
            // 將拖拽元素暫時變為半透明
            viewHolder?.itemView?.alpha = 0.7f
        }
    }

    override fun clearView(
        recyclerView: RecyclerView,
        viewHolder: RecyclerView.ViewHolder
    ) {
        super.clearView(recyclerView, viewHolder)
        // 拖拽完成後變回不透明
        viewHolder.itemView.alpha = 1.0f
    }
})

```

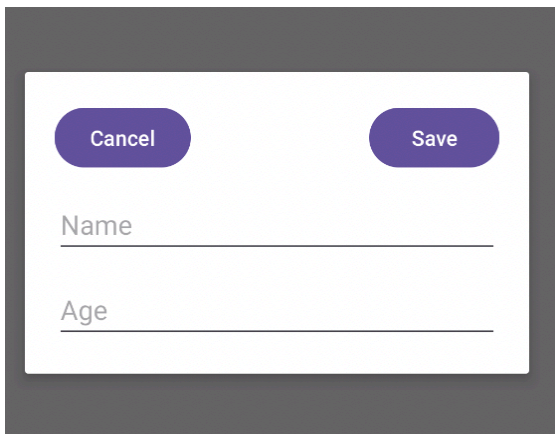
2. 把 itemTouchHelper 加到 RecyclerView 中

```

itemTouchHelper.attachToRecyclerView(recyclerView)

```

自定義 AlertDialog 中內容



1. 建立 AlertDialog 中 layout file

custom_dialog_layout.xml

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:padding="20dp"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
.....

</LinearLayout>
```

2. 讀入模版

```
val customView = LayoutInflater.inflate(R.layout.custom_dialog_layout,
null)
val saveButton = customView.findViewById<Button>(R.id.saveButton)
val cancelButton = customView.findViewById<Button>(R.id.cancelButton)

val alertDialog = AlertDialog.Builder(this).setView(customView).show()

cancelButton.setOnClickListener {
    alertDialog.dismiss()
}

saveButton.setOnClickListener {
    Log.d("tag", "Save Clicked")
}
```